



Entrega Final de la Aplicación EntregaYa

Autores

Andrés Castrillo, Esteban Chaustre, Alejandro Martinez, Daniel Barreto, Juan Arbelaez

Ingeniería de Sistemas

Presentado a: Luis Moreno Sandoval

Fecha de entrega: 19 de mayo 2026

1. Introducción

El desarrollo de software moderno requiere no solo la implementación de funcionalidades, sino también la aplicación de estrategias que garanticen altos estándares de calidad, seguridad, mantenibilidad y rendimiento. En este contexto, el proyecto EntregaYa incorporó un conjunto de herramientas y prácticas orientadas al aseguramiento continuo de la calidad del sistema, integrando procesos automatizados de análisis estático, pruebas unitarias, cobertura de código, despliegue continuo y monitoreo de rendimiento dentro de una arquitectura basada en integración continua y metodologías ágiles.

Para lograrlo, se implementó una pipeline de CI/CD utilizando GitHub Actions, la cual permitió automatizar procesos fundamentales como la compilación del proyecto, ejecución de pruebas, análisis de métricas, construcción de imágenes Docker y despliegue continuo. Dentro de este flujo se integraron herramientas como SonarQube, JaCoCo y Checkstyle, complementadas con métricas personalizadas de latencia, permitiendo evaluar aspectos relacionados con seguridad, fiabilidad, cobertura de pruebas, estandarización del código y desempeño del sistema.

Adicionalmente, el proyecto incorporó tecnologías de contenedorización mediante Docker y Docker Compose para garantizar portabilidad y consistencia entre entornos, así como la integración de servicios externos como Twilio para el envío automatizado de notificaciones SMS. Todo este proceso estuvo acompañado por la aplicación de metodologías ágiles y prácticas colaborativas de desarrollo, permitiendo mantener trazabilidad, control de versiones y mejora continua a lo largo de cada sprint.

El presente documento describe las principales métricas de calidad implementadas, las problemáticas identificadas durante el desarrollo, las soluciones aplicadas por el equipo y el impacto de dichas mejoras sobre la robustez, mantenibilidad y estabilidad del sistema. Asimismo, se presentan los resultados obtenidos en términos de cobertura, calidad de código, automatización y rendimiento, evidenciando el cumplimiento de los objetivos técnicos establecidos para la entrega final del proyecto.

2. Implementación Métricas de Calidad

2.1. Métricas de Calidad de SonarQube

SonarQube evalúa distintos aspectos relacionados con la calidad y mantenibilidad del código alojado en un repositorio de GitHub. Para integrarlo al proyecto trabajado, fue necesario implementar una serie de acciones previas. Entre ellas, y con el apoyo del monitor, se desarrolló un pipeline que permitió vincular SonarQube directamente con el repositorio de la organización de la clase *puj-course*, como se muestra en la siguiente figura:

 **JuanParias29** Add SonarQube workflow for build and analysis  b7b096a · last week 

5 lines (36 loc) · 1.32 KB

CodeBlame

RawCopyDownloadEditDropdownCompare

```
1  name: SonarQube
2  on:
3    push:
4      branches:
5        - main
6    pull_request:
7      types: [opened, synchronize, reopened]
8  jobs:
9    build:
10     name: Build and analyze
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@34e114876b0b11c390a56381ad16ebd13914f8d5 # v4.3.1
14         with:
15           fetch-depth: 0 # Shallow clones should be disabled for a better relevancy of a
```

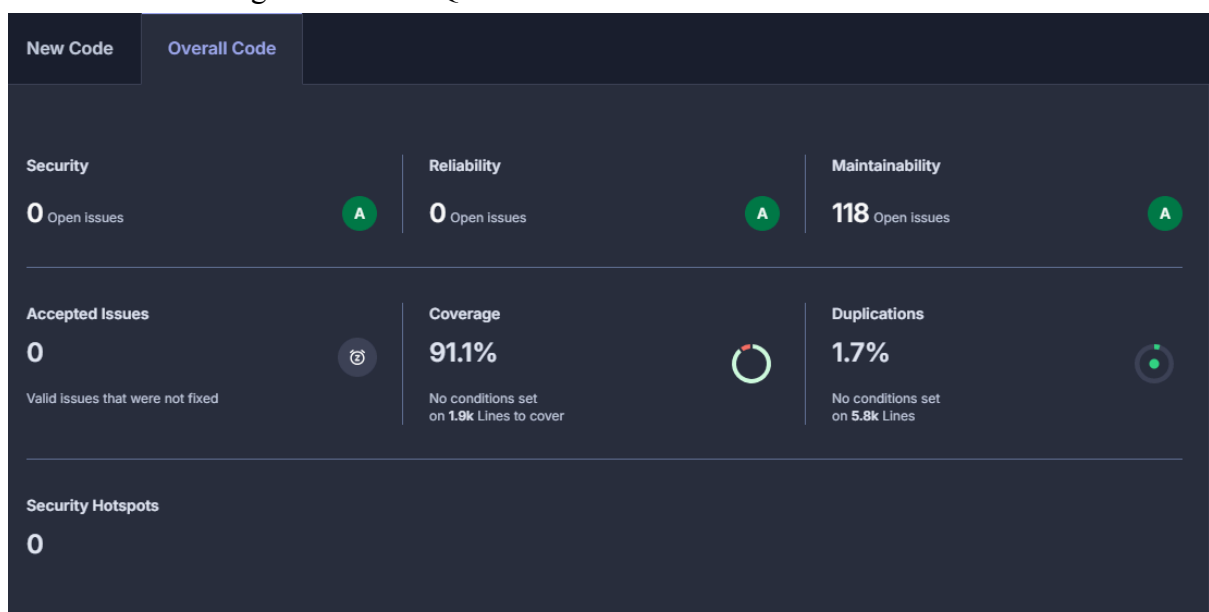
Posteriormente, se integraron las pruebas ejecutadas por SonarQube al pipeline final de integración continua, cuyas especificaciones se detallarán más adelante. A continuación, se presenta la fase cinco de dicho pipeline, en la que se ejecutan las tareas correspondientes al análisis y depuración mediante esta herramienta:

```
← Files main FIS_2610_3513_G3 / .github / workflows / CI-pruebas.yml ↑ Top

Code Blame Raw Copy Download Edit Dropdown Code

169
170 # STAGE 5: SonarQube Quality Gate
171 sonarqube:
172   name: "Stage 5 - SonarQube Quality Gate"
173   runs-on: ubuntu-latest
174   needs: static-analysis
175
176   steps:
177     - uses: actions/checkout@v4
178       with:
179         fetch-depth: 0
180
181     - name: Configurar Java 17
182       uses: actions/setup-java@v4
183       with:
184         java-version: '17'
185         distribution: 'temurin'
186         cache: maven
187
188     - name: Crear configuración de pruebas
```

Tras realizar múltiples correcciones en el código y agregar pruebas unitarias para cada uno de los archivos del repositorio, el equipo de trabajo logró alcanzar la meta establecida de una cobertura superior al 80% y un porcentaje de duplicación de código inferior al 3%, especificaciones que serán trabajadas en la sección 4. Adicionalmente, se mejoraron significativamente los resultados iniciales obtenidos durante el desarrollo del taller, alcanzando una calificación de A en todos los aspectos evaluados, como se evidencia en el dashboard de EntregaYa en SonarQube:

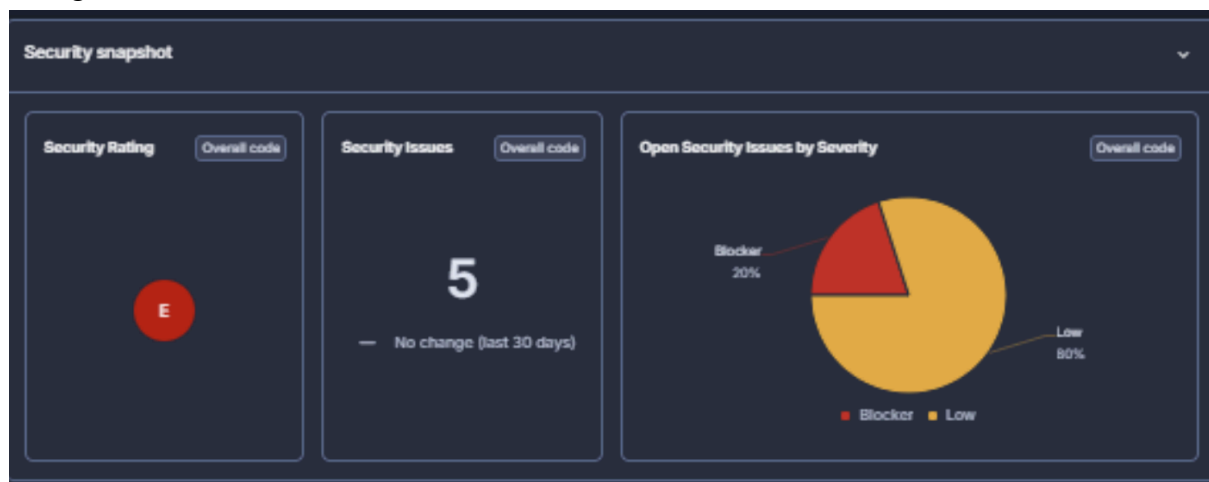


A continuación, se presentan dos de las métricas más representativas evaluadas por SonarQube, junto con su interpretación técnica, el impacto que tienen sobre la calidad del sistema y las acciones implementadas por el equipo para mejorar los resultados obtenidos durante el desarrollo del proyecto. Asimismo, se describen las problemáticas identificadas en los análisis iniciales, las soluciones aplicadas para corregirlas y la manera en que dichas mejoras contribuyeron a alcanzar estándares más altos de calidad, seguridad y mantenibilidad del software.

2.1.1. Seguridad

La métrica de seguridad en SonarQube evalúa la presencia de vulnerabilidades y prácticas inseguras dentro del código fuente y de la configuración del proyecto. Esta métrica identifica posibles riesgos que podrían comprometer la confidencialidad, integridad o disponibilidad del sistema, tales como exposición de credenciales, configuraciones inseguras, uso inadecuado de datos sensibles o vulnerabilidades conocidas.

En el primer análisis realizado por SonarQube se obtuvo una calificación de E, lo que indicaba la existencia de múltiples problemas críticos de seguridad que podían representar un riesgo importante para el proyecto. Entre las principales fallas detectadas se encontraba la exposición de credenciales sensibles directamente en el repositorio, como la contraseña de la base de datos, así como variables relacionadas con DockerHub, específicamente el token y el username, los cuales estaban definidos de manera insegura dentro del código o archivos de configuración.

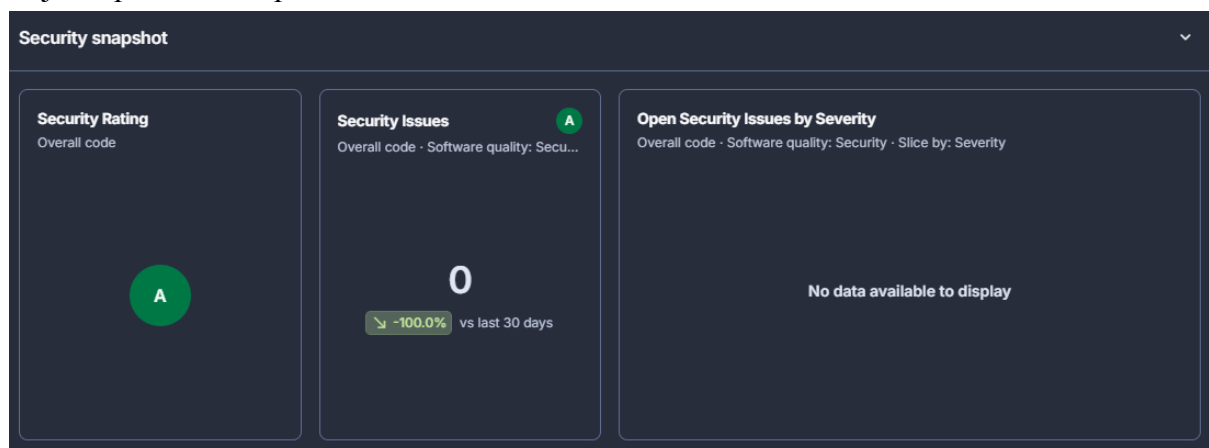


Una calificación de E en esta métrica implica que el sistema presenta vulnerabilidades graves que podrían ser explotadas por terceros, afectando la seguridad de la aplicación y comprometiendo información sensible. Este tipo de problemas reduce significativamente la confiabilidad del sistema y dificulta su despliegue en ambientes productivos.

Para solucionar estas vulnerabilidades, el equipo migró todas las credenciales sensibles al sistema de GitHub Secrets, evitando que la información confidencial quedará expuesta en el repositorio. Además, se realizaron ajustes en la configuración del pipeline para acceder de manera segura a dichas variables durante los procesos de integración continua y despliegue.

Repository secrets			New repository secret	
Name 			Last updated	
 DB_NAME			last week	 
 DB_PASSWORD			last week	 
 DB_USER			last week	 
 DOCKERHUB_TOKEN			last week	 
 DOCKERHUB_USERNAME			last week	 
 GH_PAT			last month	 
 SONARCLOUD_TOKEN			5 days ago	 
 SONAR_TOKEN			last week	 
 TELEGRAM_BOT_TOKEN			last week	 
 TELEGRAM_CHAT_ID			last week	 

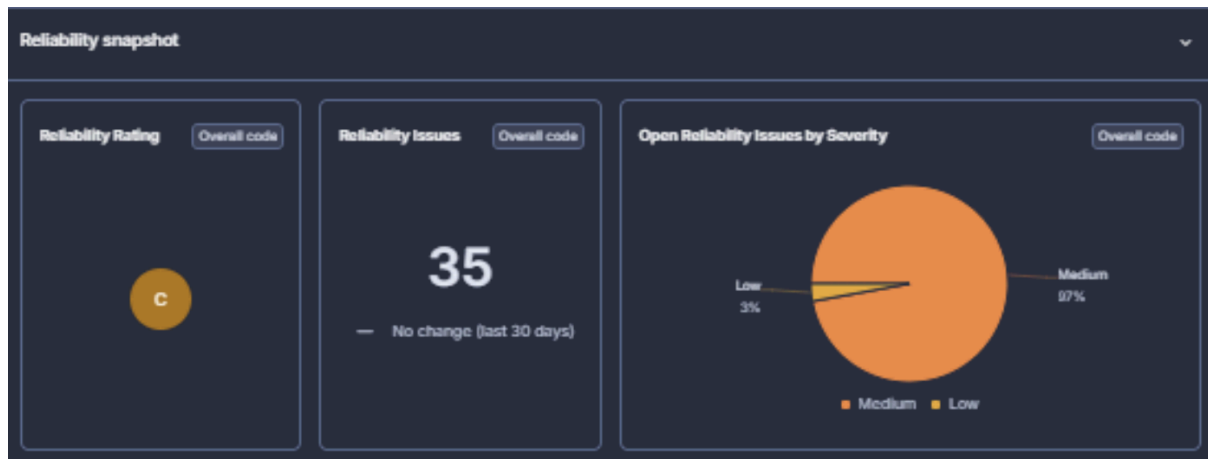
Gracias a estas correcciones, el proyecto alcanzó una calificación de A en seguridad, evidenciando que las vulnerabilidades críticas fueron mitigadas y que el sistema ahora sigue mejores prácticas de protección de información sensible.



2.1.2. Fiabilidad

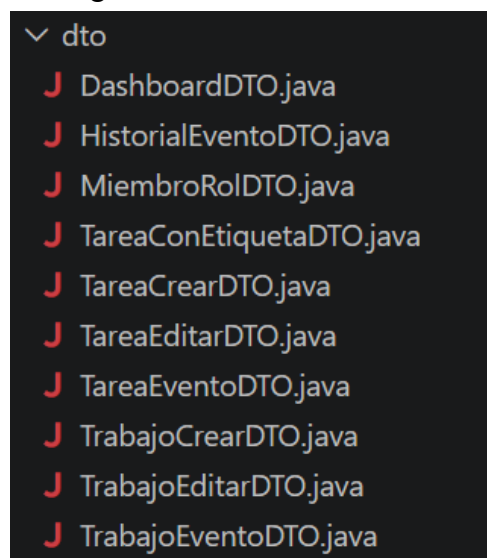
La métrica de fiabilidad (Reliability) evalúa la capacidad del sistema para funcionar correctamente sin generar errores, fallos inesperados o comportamientos incorrectos durante su ejecución. SonarQube analiza aspectos relacionados con bugs, malas prácticas de programación y estructuras de código que puedan producir inconsistencias o errores lógicos. En el análisis inicial, esta métrica obtuvo una calificación de C, debido principalmente a que se estaban generando múltiples copias de las entidades Tarea y Trabajo directamente desde las clases del modelo. Esta práctica ocasiona problemas de mantenimiento y aumentaba el

riesgo de inconsistencias en los datos, además de generar advertencias relacionadas con duplicación de lógica y manipulación inadecuada de objetos.

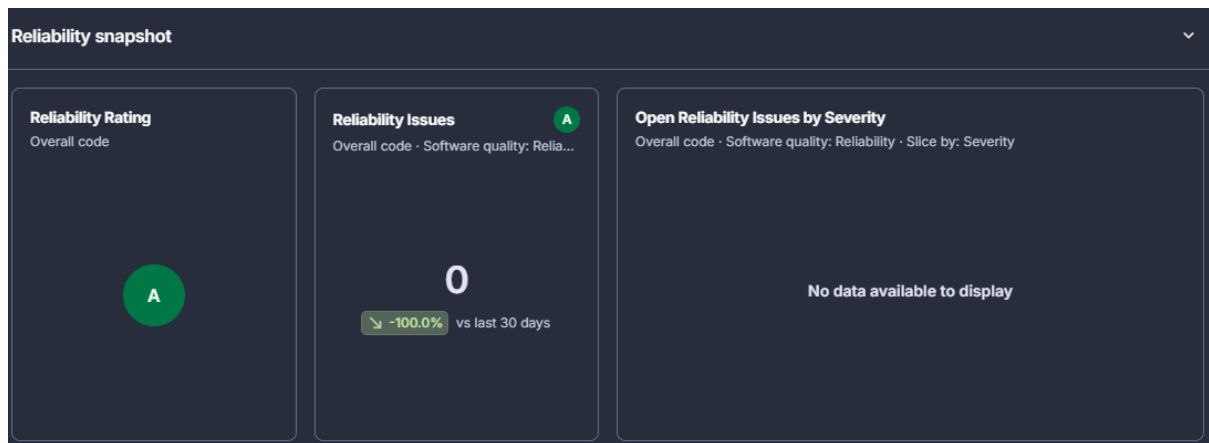


Una calificación de C indica que existen problemas relevantes de fiabilidad que, aunque no necesariamente provocan fallos críticos inmediatos, sí incrementan la probabilidad de errores en el sistema y afectan negativamente la estabilidad del código a largo plazo. Este tipo de resultados evidencia la necesidad de aplicar mejoras estructurales para reducir riesgos y facilitar el mantenimiento del software.

Para corregir esta situación, fue necesario implementar clases DTO para las entidades Tarea y Trabajo. A través de estas clases se centralizó la transferencia de datos entre capas del sistema, evitando la manipulación directa de las entidades del modelo y reduciendo la duplicación de lógica. Esta solución permite desacoplar mejor la arquitectura de la aplicación y mejorar la organización del código.



Como resultado de estas mejoras, la métrica de fiabilidad alcanzó una calificación de A, reflejando una reducción significativa de problemas potenciales y una estructura de código más robusta, mantenible y estable.



Aunque la solución implementada resolvió el problema identificado, otras acciones adicionales que podrían fortalecer aún más esta métrica requieren implementar validaciones más estrictas sobre los datos recibidos y realizar revisiones periódicas de código enfocadas en detección temprana de errores. Estas acciones contribuirían a disminuir aún más la probabilidad de fallos y mejorar la calidad general del sistema.

2.2. Métricas adicionales a SonarQube

Con el objetivo de ampliar la validación de la calidad del proyecto mediante métricas complementarias a las evaluadas previamente con SonarQube, se implementaron herramientas adicionales orientadas al análisis de cobertura, estandarización del código y rendimiento del sistema. En este contexto, se integraron tres métricas principales: JaCoCo, Checkstyle y latencia.

La incorporación de estas métricas permitió obtener una evaluación más integral del estado del proyecto, abordando no solo aspectos relacionados con seguridad, mantenibilidad y fiabilidad, sino también la calidad de las pruebas realizadas, el cumplimiento de buenas prácticas de programación y el comportamiento del sistema frente a tiempos de respuesta. Gracias a ello, fue posible identificar oportunidades de mejora adicionales y fortalecer la robustez general de la aplicación.

2.2.1. Métricas de JaCoCo

JaCoCo es una herramienta de análisis de cobertura de código que mide qué porcentaje del código fuente es ejecutado durante la ejecución de las pruebas unitarias. Se integró en la pipeline de integración continua como el Stage 3, ejecutándose automáticamente después de las pruebas unitarias mediante el comando `mvn verify`, y generando un reporte HTML descargable como artefacto del pipeline en GitHub Actions.


```
# STAGE 3: Cobertura de Código (JaCoCo)
coverage:
  name: "Stage 3 - Cobertura (JaCoCo)"
  runs-on: ubuntu-latest
  needs: test

  steps:
    - uses: actions/checkout@v4

    - name: Configurar Java 17
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: maven

    - name: Crear configuración de pruebas
      run: |
        mkdir -p src/test/resources

        cat > src/test/resources/application-test.properties << 'PROPS'
        spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
        spring.datasource.driver-class-name=org.h2.Driver
        spring.datasource.username=sa
        spring.datasource.password=
        spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
        spring.jpa.hibernate.ddl-auto=create-drop
        spring.security.user.name=test
        spring.security.user.password=test
        telegram.bot.token=dummy
        telegram.bot.chat-id=dummy
        PROPS
```

```
- name: Ejecutar pruebas con cobertura JaCoCo
  run: |
    mvn verify \
      -Dspring.profiles.active=test \
      --no-transfer-progress \
      -Dsurefire.failIfNoSpecifiedTests=false

- name: Verificar umbral de cobertura
  run: |
    mvn jacoco:check \
      --no-transfer-progress \
      || echo "▲ Cobertura por debajo del umbral mínimo"

- name: Subir reporte JaCoCo
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: target/site/jacoco/
    retention-days: 30
```

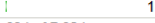
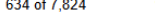
La métrica evalúa la cobertura a nivel de instrucciones, ramas, líneas, métodos y clases. Su interpretación es directa: un porcentaje alto indica que las pruebas ejercitan una proporción significativa del código, reduciendo la probabilidad de que errores pasen desapercibidos. Un porcentaje bajo, en cambio, evidencia zonas del sistema que no están siendo validadas, lo que incrementa el riesgo de fallos en producción. El equipo estableció como umbral mínimo una cobertura de instrucciones superior al 80%, verificada automáticamente mediante `mvn jacoco:check`. Si este umbral no se cumple, el pipeline reporta una advertencia y bloquea el avance a etapas posteriores.

Durante las primeras iteraciones del proyecto, la cobertura era insuficiente en varios paquetes, principalmente en los servicios y controladores, debido a que las pruebas unitarias no abarcaban todos los flujos posibles de ejecución. Esto representó un riesgo para la estabilidad del sistema, ya que la lógica de negocio crítica quedaba sin validar. Para corregir esta situación, el equipo implementó pruebas adicionales orientadas a cubrir los flujos faltantes, logrando incrementar progresivamente la cobertura hasta superar el umbral establecido.

Los resultados finales obtenidos sobre un total de 7.824 instrucciones y 56 clases fueron los siguientes: cobertura de instrucciones del 91% , cobertura de ramas del 78%, 1.751 líneas cubiertas de 1.862, 442 métodos cubiertos de 499, y las 56 clases del proyecto completamente cubiertas. Por paquete, los resultados más destacados fueron el paquete `dto` con 100% de cobertura, `observer` con 99%, `strategy` y `facade` con 96%, y `builder` con 96%. El paquete con menor cobertura fue `interceptor` con 67%, debido a la naturaleza de su integración con el contexto HTTP, que dificulta su prueba en entorno aislado.

El impacto de esta métrica sobre la calidad del sistema es significativo, ya que permite detectar zonas del código sin validar, garantizar que los cambios realizados no rompen funcionalidades existentes y fortalecer la confianza en el comportamiento del sistema antes de cada despliegue. Los resultados obtenidos superan ampliamente el umbral mínimo del 80% establecido por el equipo, evidenciando una estrategia de pruebas robusta y consistente a lo largo de todos los módulos del sistema.

entregaya

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.example.entregaya.service		90%		82%	56	222	39	652	26	124	0	8
com.example.entregaya.controller		91%		56%	36	126	27	473	12	74	0	6
com.example.entregaya.model		89%		94%	17	171	35	331	15	151	0	14
com.example.entregaya.interceptor		67%		75%	2	6	4	17	1	4	0	1
com.example.entregaya.decorator		92%		91%	3	36	3	52	2	30	0	7
com.example.entregaya.facade		96%		79%	6	23	2	61	1	11	0	1
com.example.entregaya.observer		99%		92%	4	46	1	121	0	18	0	4
com.example.entregaya.builder		96%		78%	3	16	0	30	0	9	0	1
com.example.entregaya.dto		100%		100%	0	77	0	120	0	74	0	12
com.example.entregaya.strategy		100%		83%	1	7	0	5	0	4	0	2
Total	634 of 7,824	91%	96 of 457	78%	128	730	111	1,862	57	499	0	56

2.2.1. Métrica de Checkstyle

Checkstyle es una herramienta de análisis estático que verifica que el código fuente cumpla con un conjunto de convenciones de estilo y buenas prácticas de programación. Se integró en la pipeline de integración continua como el Stage 4, ejecutándose mediante el comando `mvn checkstyle:check` y generando un reporte XML descargable como artefacto del pipeline en GitHub Actions.

```
# STAGE 4: Análisis Estático (Checkstyle)
static-analysis:
  name: "Stage 4 - Análisis Estático Checkstyle"
  runs-on: ubuntu-latest
  needs: coverage

  steps:
    - uses: actions/checkout@v4

    - name: Configurar Java 17
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: maven

    - name: Ejecutar Checkstyle
      run: |
        mvn checkstyle:check \
          --no-transfer-progress \
          || echo "▲ Checkstyle encontró problemas"

    - name: Subir reporte Checkstyle
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: checkstyle-report
        path: target/checkstyle-result.xml
        retention-days: 30
```

La métrica evalúa aspectos como la nomenclatura de variables, clases y métodos, el uso correcto de espacios e indentación, la longitud máxima de líneas, la organización de imports y la presencia de comentarios de documentación. Su interpretación es clara: cada violación detectada representa una inconsistencia con las convenciones de escritura acordadas por el

equipo, lo que afecta la legibilidad, uniformidad y mantenibilidad del código. Un resultado sin errores indica que todo el código del proyecto sigue los estándares de estilo definidos.

En el análisis inicial, Checkstyle detectó 58 errores distribuidos a lo largo del código fuente del proyecto, relacionados principalmente con problemas de formato, longitud de líneas que superan el límite permitido, convenciones de nombrado inconsistentes y ausencia de comentarios de documentación en algunos métodos. Cada uno de estos errores representó una violación a las reglas de estilo configuradas, lo que evidenciaba la necesidad de una revisión sistemática del código para garantizar su uniformidad.

Para corregir estas violaciones, el equipo realizó una revisión sistemática de las clases afectadas, ajustando el formato del código, reorganizando los imports, corrigiendo la longitud de líneas que superan el límite establecido y estandarizando los nombres de variables y métodos según las convenciones configuradas. Esta actividad permitió no sólo eliminar los errores reportados, sino también mejorar la coherencia visual del código a lo largo de todo el repositorio.

El impacto de esta métrica sobre la calidad del sistema es relevante, ya que un código con estándares de estilo consistentes facilita la revisión entre compañeros, reduce la curva de aprendizaje para nuevos integrantes y disminuye la probabilidad de introducir errores por falta de claridad en la lectura del código. Como resultado del proceso de corrección, el análisis final de Checkstyle no reportó ningún error, logrando un cumplimiento del 100% de las reglas de estilo configuradas.

```
[INFO]
[INFO] -----< com.example:entregaya >-----
[INFO] Building entregaya 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- checkstyle:3.6.0:check (default-cli) @ entregaya ---
[INFO] Comenzando auditoría...
Auditoría concluida.
[INFO] You have 0 Checkstyle violations.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.023 s
[INFO] Finished at: 2026-05-17T21:42:41-05:00
[INFO] -----
```

Como posibles acciones adicionales de mejora, se podría incorporar la verificación de Checkstyle directamente en los editores de código de cada integrante mediante plugins, de modo que los errores sean detectados y corregidos en tiempo real durante el desarrollo, antes de llegar al pipeline de integración continua. Esto reduciría el tiempo de corrección y garantiza el cumplimiento de los estándares de manera continua.

2.2.3. Métrica de Latencia

Con el propósito de complementar la validación de calidad del sistema mediante métricas de rendimiento, se implementó una métrica personalizada de latencia utilizando un interceptor denominado LatenciaInterceptor junto con pruebas automatizadas desarrolladas con MockMvc. Esta métrica permite medir el tiempo de respuesta de distintos endpoints críticos

de la aplicación, entre ellos /login, /dashboard y /trabajos, con el fin de identificar posibles problemas de rendimiento antes del despliegue en producción.

```
@Component
public class LatenciaInterceptor implements HandlerInterceptor {

    private static final Logger log = LoggerFactory.getLogger(LatenciaInterceptor.class);

    /** Atributo usado para almacenar el timestamp de inicio en el request. */
    private static final String ATTR_INICIO = "latencia_inicio_ms";

    /** Umbral en milisegundos a partir del cual se emite una advertencia. */
    public static final long UMBRAL_MS = 1000L;

    /**
     * Se ejecuta ANTES de que el controlador procese la petición.
     * Guarda el timestamp de inicio como atributo del request.
     */
}
```

La métrica de latencia mide específicamente el tiempo que tarda el sistema en procesar una solicitud HTTP y generar una respuesta. Para ello, la implementación utiliza herramientas y componentes como HandlerInterceptor, System.currentTimeMillis(), SpringBootTest y MockMvc, integrados directamente dentro del pipeline de integración continua mediante GitHub Actions. Adicionalmente, la métrica genera automáticamente un reporte con las latencias calculadas para cada prueba ejecutada, permitiendo visualizar el comportamiento individual de cada endpoint evaluado y facilitando el análisis del rendimiento general del sistema.

```
# HU-50: Test de latencia
# Si algún endpoint supera 500 ms,
- name: Ejecutar pruebas (incluye test de latencia)
  run: |
    mvn test \
      -Dspring.profiles.active=test \
      --no-transfer-progress \
      -Dsurefire.failIfNoSpecifiedTests=false

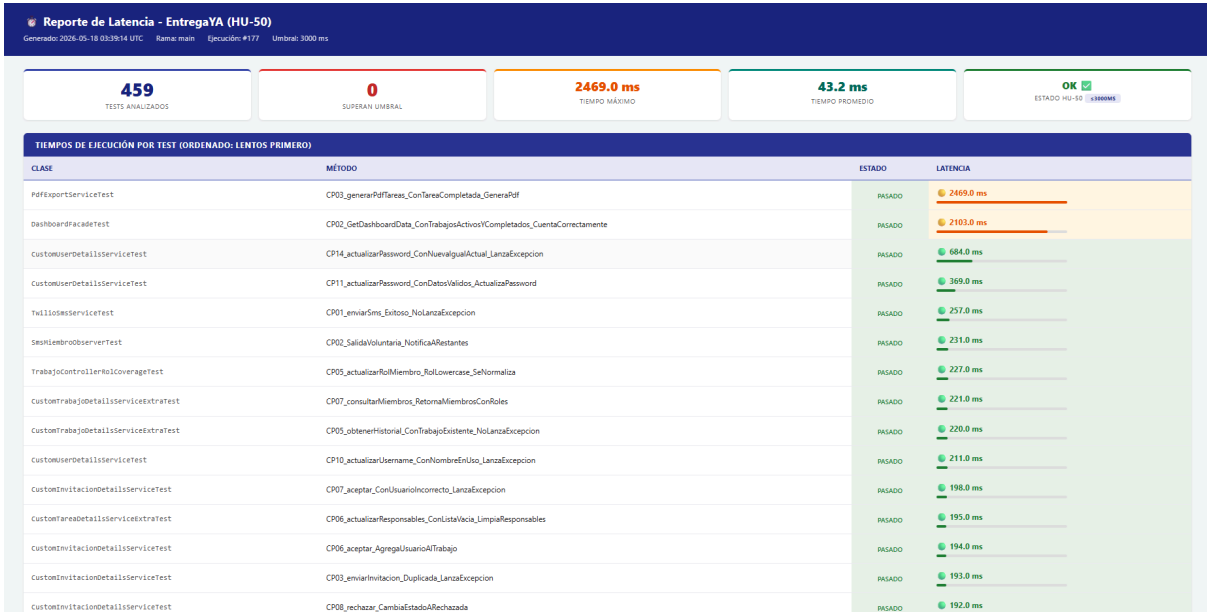
- name: Generar reporte HTML del plan de pruebas
  if: always()
  run: python3 .github/scripts/generar_reporte.py
```

La interpretación de esta métrica se basa en la comparación entre los tiempos de respuesta obtenidos y un umbral máximo permitido definido por el equipo. Un resultado por debajo del umbral indica que el endpoint responde de manera adecuada bajo las condiciones de prueba establecidas, mientras que un tiempo superior refleja posibles problemas de rendimiento que pueden afectar la experiencia del usuario y la estabilidad del sistema. En términos prácticos, una latencia elevada puede estar relacionada con consultas SQL ineficientes, exceso de procesamiento en la lógica de negocio, sobrecarga generada por componentes de seguridad o limitaciones de infraestructura en el entorno de ejecución.

```
12018 [INFO]
12019 Error: Failures:
12020 Error: LatenciaInterceptorTest.trabajosDebeResponderDentroDelUmbral:68 El endpoint /trabajos tardó 532 ms, superando el umbral de 500 ms
=> expected: <true> but was: <false>
12021 [INFO]
12022 Error: Tests run: 405, Failures: 1, Errors: 0, Skipped: 0
12023 [INFO]
12024 [INFO] -----
12025 [INFO] BUILD FAILURE
```

El impacto de esta métrica sobre la calidad del sistema es significativo, ya que permite detectar degradaciones de rendimiento de manera temprana, validar automáticamente los tiempos de respuesta y prevenir que endpoints con problemas de desempeño lleguen a

ambientes productivos. Además, fortalece el proceso de integración continua al incorporar pruebas de rendimiento automatizadas dentro del flujo de desarrollo.



Como posibles acciones adicionales de mejora, el equipo podría optimizar consultas SQL complejas, implementar mecanismos de caché, reducir operaciones innecesarias dentro de los controladores y aplicar monitoreo continuo en ambientes productivos para obtener métricas más precisas bajo carga real. Estas mejoras contribuirían a reducir aún más los tiempos de respuesta y aumentar la escalabilidad del sistema.

3. Despliegue y configuración

Para el despliegue del sistema se implementó una estrategia basada en Docker y automatización mediante GitHub Actions, permitiendo estandarizar la ejecución del backend y facilitar el despliegue continuo del proyecto.

3.1. Docker y Dockerfile

El proyecto cuenta con tres Dockerfiles, cada uno con un propósito específico. El Dockerfile principal maneja el backend en Spring Boot usando una construcción en dos etapas: una para compilar el proyecto con Maven y otra para ejecutarlo con una imagen mínima de Java, reduciendo el tamaño final de la imagen y optimizando el uso de memoria dentro del contenedor. El Dockerfile.frontend sigue la misma lógica pero termina sirviendo los archivos estáticos de la aplicación web a través de Nginx. El Dockerfile.tests está dedicado exclusivamente a correr las pruebas del proyecto en un entorno aislado, separado completamente del backend en producción.

Los tres incluyen healthchecks para que Docker pueda verificar si cada servicio está funcionando correctamente antes de que otros servicios dependan de él.

La utilización de Docker permitió:

- Ejecutar el sistema de manera consistente en diferentes entornos.
- Evitar problemas de configuración local.
- Facilitar la portabilidad del backend.
- Simplificar el proceso de despliegue.

3.2. Docker Compose

El `docker-compose.yml` levanta cinco servicios de forma coordinada: la base de datos PostgreSQL, el backend, el frontend, un contenedor de pruebas y pgAdmin como interfaz visual para administrar la base de datos. Cada servicio se comunica únicamente con los que necesita a través de redes internas segmentadas, y los datos de la base de datos y de pgAdmin se persisten mediante volúmenes. Todo el entorno completo se levanta con un único comando.

3.3. Pipeline de Despliegue Continuo (CD)

El proyecto implementó una pipeline de despliegue continuo utilizando GitHub Actions. Esta pipeline automatiza el proceso de construcción y publicación del sistema cada vez que se realizan integraciones importantes en el repositorio.

La pipeline realiza automáticamente:

- Descarga del código fuente.
- Compilación del proyecto con Maven.
- Construcción de la imagen Docker.
- Publicación de la imagen en DockerHub.
- Validación del despliegue.
- Envío de notificaciones mediante Telegram.

Gracias a esta automatización se logró reducir errores manuales en el proceso de despliegue y mantener versiones actualizadas y reproducibles del sistema.

4. Pruebas Unitarias

Para garantizar la calidad del software se implementó una estrategia de integración continua y pruebas automatizadas utilizando GitHub Actions, JUnit 5, JaCoCo y SonarQube.

4.1. Pipeline de Integración Continua (CI)

Se configuró una pipeline de integración continua mediante GitHub Actions para automatizar la validación del proyecto en cada integración de código.

La pipeline ejecuta automáticamente:

- Compilación del proyecto.
- Ejecución de pruebas unitarias.
- Generación de reportes de cobertura.
- Análisis estático del código.
- Validación de métricas de calidad.

Esto permitió detectar errores tempranamente y mantener estabilidad en el repositorio principal durante el desarrollo colaborativo.

4.2. Pruebas unitarias

Las pruebas unitarias fueron implementadas utilizando JUnit 5 para validar el comportamiento de distintos componentes del backend.

Se realizaron pruebas sobre:

- Controladores REST
- Servicios de negocio
- Validaciones principales
- Componentes críticos del sistema

Estas pruebas permitieron verificar el correcto funcionamiento de funcionalidades relacionadas con autenticación, trabajos, tareas, notificaciones y operaciones principales del sistema.

4.3. Cobertura de código con JaCoCo

Para medir la cobertura de pruebas unitarias se integró JaCoCo dentro de la pipeline de integración continua del proyecto.

Uno de los requisitos definidos para la entrega final consistía en alcanzar una cobertura mínima superior al 80% del código del sistema. Como resultado del proceso de pruebas y aseguramiento de calidad implementado por el equipo, se logró superar ampliamente este objetivo, alcanzando una cobertura general del 91.1% verificada mediante SonarQube.

Además, durante la ejecución de la pipeline CI se validó exitosamente la ejecución de todas las pruebas unitarias desarrolladas, obteniendo los siguientes resultados:

- 464 pruebas ejecutadas
- 0 fallos
- 0 errores
- 0 pruebas omitidas


```
15941 [INFO] Tests run: 464, Failures: 0, Errors: 0, Skipped: 0
15942 [INFO]
15943 [INFO] -----
15944 [INFO] BUILD SUCCESS
15945 [INFO] -----
15946 [INFO] Total time: 42.651 s
```

Estos resultados permitieron garantizar un alto nivel de confiabilidad y estabilidad del sistema antes de cada integración y despliegue.

4.4. Cobertura de código y análisis con SonarQube

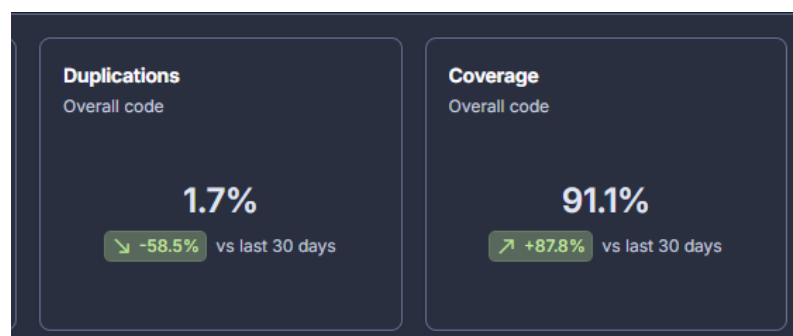
Se utilizó SonarQube como herramienta principal de análisis estático para evaluar continuamente la calidad y mantenibilidad del código fuente del proyecto. La integración de SonarQube dentro de la pipeline CI permitió analizar automáticamente aspectos como:

- Bugs potenciales
- Vulnerabilidades de seguridad
- Código duplicado
- Problemas de mantenibilidad
- Code smells
- Cobertura de pruebas unitarias

Adicionalmente, se contaba con un Quality Gate encargado de validar automáticamente si el proyecto cumplía con los estándares mínimos de calidad definidos por el equipo antes de permitir nuevas integraciones.

Como resultado del proceso de mejora continua realizado durante el desarrollo, el proyecto logró aprobar satisfactoriamente el Quality Gate de SonarQube, alcanzando los siguientes resultados finales:

- Quality Gate: Passed
- Cobertura de código: 91.1%
- Duplicación de código: 1.7%
- Reducción significativa de issues abiertos



Estos resultados reflejan una mejora importante en la calidad general del software, fortaleciendo la mantenibilidad, confiabilidad y estabilidad del sistema desarrollado.

5. Integración servicio SMS

Como parte de la entrega final del proyecto EntregaYa, se implementó la integración con el proveedor de servicios SMS Twilio, con el objetivo de notificar a los miembros de un trabajo cuando ocurren eventos relevantes, como el ingreso o la salida de un colaborador. Esta funcionalidad complementa la arquitectura de notificaciones del sistema, la cual está basada en el patrón de diseño Observer.

5.1. Arquitectura de la integración

La implementación se compone de dos clases principales. La clase TwilioSmsService actúa como servicio encargado de inicializar el cliente de Twilio mediante la anotación `@PostConstruct` y de exponer el método `enviarSms`, que construye y envía el mensaje a través de la API de Twilio usando el SDK oficial para Java. La clase `SmsMiembroObserver` implementa la interfaz `TrabajoObserver` y actúa como observador dentro del patrón de notificaciones del sistema. Al recibir un evento de tipo `TrabajoEventoDTO`, este componente itera sobre los colaboradores del trabajo afectado, filtra al usuario involucrado en el evento y envía un SMS personalizado a cada miembro mediante el TwilioSmsService.

La configuración del servicio se gestiona mediante variables de entorno definidas en el archivo `application.properties`, lo que permite desacoplar las credenciales del código fuente y seguir buenas prácticas de seguridad. Las propiedades configuradas son: `twilio.account.sid`, `twilio.auth.token` y `twilio.from.number`.

5.2. Problemas encontrados y solución aplicada

Durante el proceso de integración se presentaron varios inconvenientes relacionados con las restricciones de la cuenta Trial de Twilio y la configuración del número de origen. Inicialmente, los mensajes aparecían con estado `queued` en los registros del sistema y del panel de Twilio, pero nunca llegaban al destinatario. Tras el análisis de los logs y la consola de Twilio, se identificaron las siguientes causas:

- Cuenta Trial de Twilio: Las cuentas en modalidad de prueba únicamente permiten enviar SMS a números previamente verificados en la consola de Twilio. Cualquier mensaje dirigido a un número no verificado queda en estado `queued` de forma indefinida sin generar ningún error explícito.
- Permisos geográficos: Twilio requiere habilitar explícitamente los permisos de envío hacia países específicos. En este caso fue necesario activar los permisos para Colombia (+57) desde la sección Geo Permissions en la consola de Twilio.
- Filtrado por operador móvil: Aunque Twilio reportaba el estado `Delivered` para los mensajes enviados, el operador de telefonía móvil Claro Colombia aplica filtros sobre SMS internacionales de tipo A2P (aplicación a persona), lo que impide que el mensaje llegue al dispositivo del destinatario a pesar de haber sido aceptado por la red. Este comportamiento es independiente del código y de la configuración del proveedor.

Para solucionar los dos primeros problemas, se verificó el número de destino en la sección Verified Caller IDs de la consola de Twilio y se habilitaron los permisos geográficos para Colombia. Con estas acciones, los mensajes pasaron a reportarse como Delivered en el panel de Twilio, confirmando que el sistema realiza correctamente el envío. El filtrado de Claro Colombia es una limitación del operador de telecomunicaciones, ajena al sistema desarrollado, y se verificó que con números de otros operadores el mensaje sí llega correctamente al dispositivo.

5.3. Evidencia del funcionamiento

El funcionamiento de la integración se evidencia mediante dos fuentes de registro complementarias. En primer lugar, los logs del servidor muestran el SID del mensaje generado por Twilio y el estado inicial del envío:

```
INFO 24180 --- [entregaya] [nio-8081-exec-6] c.e.entregaya.service.TwilioSmsService
: [TwilioSMS] SMS enviado a=+573160564285 sid=SM19028b7c1fe09af8f60f0832b9977f1e
estado=queued
```

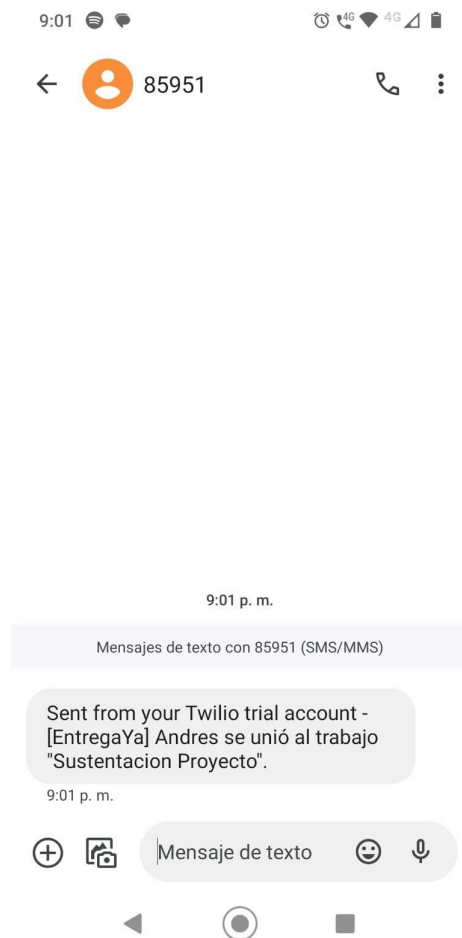
```
INFO 24180 --- [entregaya] [nio-8081-exec-6] c.e.e.observer.SmsMiembroObserver
: [SmsMiembro] SMS enviado a usuario=Daniel
```

En segundo lugar, el panel de mensajes de Twilio registra los envíos realizados con estado Delivered, confirmando que los mensajes fueron aceptados y entregados a la red del operador de destino. A continuación se presenta el registro de los mensajes enviados durante las pruebas del sistema:

DATE 	SERVICE	DIRECTION	FROM	TO	BODY 	# SEGMENTS	STATUS	MEDIA
2026-05-17 19:06:44 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 3124617524		2	 Delivered	-
2026-05-17 19:05:48 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 326104		1	 Failed  Troubleshoot	-
2026-05-17 19:01:36 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 3160564285		2	 Delivered	-
2026-05-17 18:43:21 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 31032241762		1	 Failed  Troubleshoot	-
2026-05-17 18:12:20 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 3229417591		2	 Delivered	-
2026-05-16 14:26:19 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 3229417591		2	 Delivered	-
2026-05-16 14:25:05 GMT-7	-	Outgoing API	(US) +1 2543484864	(CO) +57 3229417591		2	 Delivered	-

En la imagen observamos 2 fallos de envío, estos fueron porque el número estaba mal escrito y porque el número no estaba verificado, como evidenciamos las que no tienen esta problemática se visualizan correctamente enviados.

Estos registros demuestran que la integración con Twilio está correctamente implementada y es funcional y comprobable, cumpliendo el criterio establecido para este punto de la entrega. Para confirmar el funcionamiento del microservicio, se realizó una prueba con un número de un operador diferente a Claro. Como se observa en la siguiente figura, el mensaje SMS llegó exitosamente al dispositivo.



Este resultado confirma al 100% que la integración con Twilio está correctamente implementada y que el microservicio de notificaciones SMS funciona de forma funcional y comprobable. El prefijo "Sent from your Twilio trial account" es propio de las cuentas en modalidad Trial del proveedor y desaparece al usar una cuenta de producción.

6. Metodologías ágiles y posmortem

El documento correspondiente a la aplicación de metodologías ágiles y al análisis postmortem del proyecto se encuentra en el Anexo 1. En este se evidencia una implementación rigurosa y consistente de la metodología ágil empleada, respaldada por un reporte estructurado y cuantitativo que demuestra el cumplimiento de los milestones definidos según la planificación inicial, así como la finalización y trazabilidad temporal de todas las historias de usuario junto con sus respectivas características y criterios asociados.

Asimismo, el documento incluye el análisis del número de historias de usuario cerradas por sprint y el promedio de HU completadas en cada iteración, además del número de commits realizados por sprint y el promedio de contribuciones efectuadas por el equipo, evidenciando una distribución equilibrada del trabajo entre los integrantes. También se presenta evidencia clara de la participación de cada miembro mediante commits, pull requests y revisiones de código, junto con un control de versiones organizado a través del manejo de ramas y releases, garantizando la trazabilidad de los cambios realizados en cada versión del proyecto.

Adicionalmente, el análisis postmortem desarrolla una reflexión profunda tanto a nivel técnico como de procesos, identificando las principales causas raíz de los problemas

encontrados durante el desarrollo y proponiendo acciones concretas de mejora orientadas a optimizar futuras iteraciones del proyecto.

7. Conclusiones

El desarrollo del proyecto EntregaYa permitió evidenciar la importancia de integrar prácticas de aseguramiento de calidad, automatización y análisis continuo dentro del ciclo de desarrollo de software. La implementación de herramientas como SonarQube, JaCoCo, Checkstyle y métricas de latencia facilitó la identificación temprana de vulnerabilidades, problemas de mantenibilidad, errores potenciales y deficiencias en cobertura de pruebas, permitiendo aplicar soluciones oportunas que fortalecieron significativamente la calidad general del sistema.

Uno de los principales logros obtenidos fue la integración completa de una pipeline de CI/CD mediante GitHub Actions, capaz de automatizar procesos de compilación, pruebas unitarias, análisis estático, construcción de imágenes Docker y despliegue continuo. Esta automatización redujo errores manuales, mejoró la estabilidad del repositorio y garantizó validaciones constantes sobre cada integración realizada por el equipo de desarrollo.

En términos de calidad de código, el proyecto alcanzó resultados altamente satisfactorios. Se logró superar ampliamente el umbral de cobertura definido inicialmente, alcanzando una cobertura superior al 91%, además de reducir la duplicación de código a menos del 2% y obtener calificaciones A en las métricas principales evaluadas por SonarQube, incluyendo seguridad y fiabilidad. Estos resultados reflejan una arquitectura más robusta, desacoplada y mantenible, apoyada por buenas prácticas de programación y una estrategia sólida de pruebas automatizadas.

Asimismo, la integración de Docker y Docker Compose permitió estandarizar el entorno de ejecución del sistema y facilitar tanto el despliegue como la portabilidad del proyecto. De igual forma, la incorporación del servicio SMS mediante Twilio demostró la capacidad del sistema para integrarse con servicios externos reales, ampliando las funcionalidades de notificación implementadas mediante el patrón Observer.

Finalmente, la aplicación de metodologías ágiles y el análisis postmortem contribuyeron a mejorar la organización del equipo, la trazabilidad del trabajo y la capacidad de adaptación frente a problemas técnicos y de gestión encontrados durante el desarrollo. En conjunto, todas las prácticas y herramientas implementadas permitieron construir un sistema funcional, estable y alineado con estándares modernos de desarrollo de software, fortaleciendo tanto las competencias técnicas del equipo como la calidad final del producto entregado.